

Actividad Resuelta: Optimización con Recocido Simulado (*Simulated Annealing*) en MATLAB

Autor: Prof. D.Sc. BARSEKH-ONJI Aboud

Institución: Facultad de Ingeniería, Universidad Anáhuac México

Contacto: aboud.barsekh@anahuac.mx | ORCID: 0009-0004-5440-8092

Curso: Fundamentos de Inteligencia Artificial

Herramienta: MATLAB — Live Task *Optimize* (simulannealbnd)

Objetivo de la actividad

Aplicar el algoritmo de **Recocido Simulado** (*Simulated Annealing*, SA) para encontrar el mínimo global de una función multivariable no lineal con restricciones de caja (*box constraints*), utilizando la interfaz visual *Optimize* de MATLAB Live Editor, sin necesidad de escribir código adicional.

1. Introducción: ¿Qué es el Recocido Simulado?

El **Recocido Simulado** es un algoritmo metaheurístico de optimización inspirado en el proceso físico de **recocido de metales**: al calentar un metal y enfriarlo lentamente, los átomos encuentran configuraciones de mínima energía (estados más estables).

Analogía física ↔ Optimización

Concepto físico	Equivalente en optimización
Estado del sistema	Punto candidato (x, y) en el espacio de búsqueda
Energía del sistema	Valor de la función objetivo $f(x, y)$
Temperatura T	Parámetro de control que rige la exploración
Enfriamiento	Reducción gradual de T a lo largo de las iteraciones
Estado de mínima energía	Solución óptima (mínimo global)

Regla de aceptación de Metropolis

En cada iteración, el algoritmo genera un nuevo punto candidato x' a partir del actual x :

$$P(\text{aceptar } x') = \begin{cases} 1 & \text{si } f(x') < f(x) \\ e^{-\Delta f/T} & \text{si } f(x') \geq f(x) \end{cases}$$

donde $\Delta f = f(x') - f(x)$ y T es la temperatura actual.

Nota: A diferencia de métodos de gradiente, SA *puede aceptar soluciones peores* con cierta probabilidad. Esto le permite escapar de mínimos locales. Conforme $T \rightarrow 0$, el algoritmo se vuelve más “codicioso” (*greedy*) y converge.

2. Función objetivo propuesta

Para esta actividad usaremos la siguiente función de dos variables:

$$f(x, y) = 5x^2 + 7 \cos(y)$$

Características de la función

Propiedad	Descripción
Variables	$x \in \mathbb{R}, y \in \mathbb{R}$
Término $5x^2$	Paraboloide cuadrático centrado en $x = 0$
Término $7 \cos(y)$	Función oscilatoria periódica en y (período 2π)
Tipo	No convexa en $y \rightarrow$ múltiples mínimos locales
Mínimo global analítico	$x^* = 0, y^* = \pi + 2k\pi \ (k \in \mathbb{Z}), f^* = -7$

La función es interesante porque tiene un componente suave (cuadrático en x) y un componente oscilatorio (periódico en y), lo que la hace un buen caso de prueba para SA.

Visualización conceptual

$$f(x, y) = 5x^2 + 7\cos(y)$$

Para $x = 0$: $f(0, y) = 7\cos(y) \rightarrow$ oscila entre -7 y $+7$
 Para $y = \pi$: $f(x, \pi) = 5x^2 - 7 \rightarrow$ paraboloide con mínimo en -7

El mínimo global $f^* = -7$ ocurre en $x = 0, y = \pi \approx 3.1416$.

3. Configuración en MATLAB Live Editor — Task *Optimize*

La interfaz **Optimize** es un *Live Task* que permite configurar y ejecutar problemas de optimización de forma visual, generando automáticamente el código MATLAB equivalente.

3.1 Cómo insertar el Task

1. Abrir MATLAB y crear un nuevo **Live Script**: Home → New → Live Script
 2. En el editor: Insert → Task → Optimize
 3. Se insertará el bloque interactivo en el Live Script.
-

3.2 Sección: *Create optimization variables*

Aquí se declaran las variables de decisión del problema.

Campo	Variable x	Variable y
Name	x	y
Dimensions	1×1	1×1
Type	Continuous	Continuous
Lower bound	-Inf	-Inf
Upper bound	Inf	Inf
Initial point	0	0

Nota: No se imponen restricciones de caja en este ejemplo para observar la convergencia libre del algoritmo. El punto inicial (0, 0) es arbitrario; SA explorará el espacio desde ahí.

Cómo configurarlo:

- El tipo **Continuous** indica que las variables son reales (no enteras).
 - Los campos de bounds pueden dejarse como **-Inf** / **Inf** o restringirse (por ejemplo, $x \in [-5, 5]$, $y \in [0, 2\pi]$) si se desea acotar el espacio de búsqueda.
-

3.3 Sección: *Define problem*

Campo	Configuración
Goal	Minimize (seleccionar el botón rojo)
Objective	$5*x^2 + 7*\cos(y)$
Constraints	Ninguna (dejar vacío)

Cómo ingresar la función objetivo: 1. En el menú desplegable de *Objective*, seleccionar “**Define on one line**”. 2. Escribir exactamente: $5x^2 + 7\cos(y)$

MATLAB interpreta x e y como las variables de optimización declaradas en la sección anterior. Los operadores aritméticos siguen la sintaxis estándar de MATLAB.

3.4 Sección: *Specify problem-dependent solver options*

El solucionador seleccionado es `simulannealbnd`, que es la implementación de SA con restricciones de caja de MATLAB.

Configuración recomendada para esta actividad: Algorithm settings:

Parámetro	Valor	Descripción
Initial temperature	<code>default</code>	Temperatura inicial automática (valor = 100 por defecto). Controla la amplitud de los saltos iniciales.

Run time limits:

Parámetro	Valor	Descripción
Max iterations	200	Número máximo de iteraciones del algoritmo. Para funciones simples, 200 es suficiente.

Tolerances:

Parámetro	Valor	Descripción
Function tolerance	0.0001	Criterio de parada: si el cambio en f entre iteraciones es menor que este valor, el algoritmo se detiene.

¿Por qué estos valores? Para esta función de demostración, los valores por defecto son adecuados. En aplicaciones reales (funciones costosas, alta dimensión), se aumentan las iteraciones y se ajusta el esquema de enfriamiento.

3.5 Sección: *Display progress*

Configurar el monitoreo visual del proceso de optimización:

Opción	Configuración
Text display	Final output — muestra el resumen al terminar
Best value	Activado — grafica el mejor $f(x, y)$ encontrado hasta cada iteración
Best point	Activado — grafica la posición (x, y) del mejor punto
Current value	Activado — grafica el valor actual (incluyendo soluciones aceptadas aunque sean peores)
Stopping criteria	Activado — muestra cuándo y por qué se detuvo el algoritmo
Current temperature	Activado — muestra el perfil de enfriamiento a lo largo de las iteraciones

Observación importante: La diferencia entre *Best value* y *Current value* ilustra el comportamiento estocástico de SA: el algoritmo puede moverse a soluciones peores temporalmente (*Current value* sube), pero mantiene registro del mejor encontrado (*Best value* nunca sube).

3.6 Sección: *Display results*

Activar todas las casillas para ver el reporte completo:

Resultado	Descripción
Problem	Resumen del problema configurado
Solution	Valores óptimos (x^*, y^*) encontrados
Reason solver stopped	Criterio de parada que se activó
Objective value	Valor $f(x^*, y^*)$ en la solución

4. Ejecución y resultados esperados

4.1 Cómo ejecutar

Hacer clic en el botón *(Run Section)* dentro del bloque del task, o presionar Ctrl+Enter con el cursor dentro del task.

4.2 Salida esperada en consola

Objective function value: -6.999831...

4.3 Interpretación de resultados

Variable	Valor esperado	Valor analítico
x^*	≈ 0	0 exacto
y^*	≈ 3.1416	π
$f(x^*, y^*)$	≈ -7	-7 exacto

El resultado puede variar ligeramente en cada ejecución porque SA es un algoritmo **estocástico** (aleatorio). Esto es normal y esperado.

4.4 Gráficas generadas

MATLAB generará automáticamente hasta 5 gráficas en una ventana de monitoreo:

1. **Best Function Value** — Debe decrecer monotónicamente (o quedarse igual) a lo largo de las iteraciones.
2. **Current Function Value** — Fluctúa, puede subir y bajar; muestra la exploración del espacio.
3. **Best Point** — Cómo evolucionan x^* e y^* a lo largo de las iteraciones.
4. **Current Point** — Posición actual del algoritmo en cada paso.
5. **Temperature** — Curva descendente que muestra el enfriamiento exponencial.

5. Dibujar la función objetivo

Para visualizar la función objetivo y el espacio de soluciones (contornos), se puede utilizar el siguiente código:

```
clc; clear; close all;
x = linspace(-3, 3, 300);           % 300 puntos en x   [-3, 3]
y = linspace(-2*pi, 2*pi, 300);     % 300 puntos en y   [-2, 2]
[X, Y] = meshgrid(x, y);           % Malla 2D: X e Y son matrices 300x300
F = 5*X.^2 + 7*cos(Y);
figure('Position',[50 50 900 620], 'Name','Superficie 3D');
```

```

surf(X, Y, F, 'EdgeColor','none');
colormap(jet);
colorbar;
xlabel('$x$', 'FontSize',13, 'Interpreter','latex');
ylabel('$y$', 'FontSize',13, 'Interpreter','latex');
zlabel('$f(x,y)$', 'FontSize',13, 'Interpreter','latex');
title('$f(x,y) = 5x^2 + 7\cos(y)$', ...
      'FontSize',15, 'Interpreter','latex');
view(45, 30);
grid on;
set(gca, 'FontSize',11, 'TickLabelInterpreter','latex');
hold on;
plot3(0, pi, -7, 'ro', 'MarkerSize',12, 'MarkerFaceColor','r', ...
      'DisplayName','M\''inimo global $(0,\pi,-7)$');
legend('FontSize',11, 'Interpreter','latex', 'Location','northeast');
hold off;
figure('Position',[100 100 800 560], 'Name','Mapa de Contornos');
contourf(X, Y, F, 40, 'LineColor','none');
colormap(jet);
colorbar;
hold on;
contour(X, Y, F, 12, 'LineColor','k', 'LineWidth',0.4);
plot(0, pi, 'r*', 'MarkerSize',14, 'LineWidth',2, ...
      'DisplayName','M\''inimo global $(0,\pi)$');
plot(0, -pi, 'r*', 'MarkerSize',14, 'LineWidth',2, 'HandleVisibility','off');
plot(0, 3*pi, 'r*', 'MarkerSize',14, 'LineWidth',2, 'HandleVisibility','off');
plot(0, -3*pi, 'r*', 'MarkerSize',14, 'LineWidth',2, 'HandleVisibility','off');

xline(0, 'w--', 'LineWidth',1.0);
yline(pi, 'w:', 'LineWidth',1.0);

xlabel('$x$', 'FontSize',13, 'Interpreter','latex');
ylabel('$y$', 'FontSize',13, 'Interpreter','latex');
title('Mapa de contornos - $f(x,y) = 5x^2 + 7\cos(y)$', ...
      'FontSize',14, 'Interpreter','latex');
legend('FontSize',11, 'Interpreter','latex', 'Location','southeast');
grid off;
set(gca, 'FontSize',11, 'TickLabelInterpreter','latex');
hold off;

```

Puedes copiar este código en un script `.m` estándar y ejecutarlo directamente.

6. Experimentos sugeridos

Una vez replicado el ejemplo base, se proponen las siguientes variaciones para explorar el comportamiento de SA:

Experimento A — Efecto del punto inicial

Cambiar el *Initial point* a valores distintos y observar si el resultado cambia:

Prueba	x_0	y_0	Resultado esperado
1 (base)	0	0	$f^* \approx -7$
2	3	6	$f^* \approx -7$ (SA debe escapar del mínimo local)
3	-2	9	Puede converger a $y^* = 3\pi$ (otro mínimo global)

Experimento B — Efecto de Max Iterations

Max iterations	Observación esperada
50	Puede no converger; valor mayor que -7
200 (base)	Convergencia adecuada
1000	Convergencia más robusta; mejor exploración

Experimento C — Restricciones de caja

Agregar restricciones de caja en *Create optimization variables*: - $x \in [-2, 2]$, $y \in [0, 2\pi]$

Comparar si el resultado cambia y discutir por qué.

7. Preguntas de reflexión

Responde en tu reporte de la actividad:

1. ¿Por qué SA puede encontrar el mínimo global de $f(x, y) = 5x^2 + 7\cos(y)$ mientras que un método de gradiente podría quedar atrapado en un mínimo local?
2. Observa la gráfica de **Current Function Value** vs. **Best Function Value**. ¿Qué diferencia conceptual hay entre ambas curvas y qué nos dice sobre el criterio de aceptación de Metropolis?
3. ¿En qué aplicaciones de ingeniería considerarías usar SA en lugar de métodos de gradiente como `fmincon`?
4. ¿Qué efecto tiene aumentar la **Initial Temperature** en el comportamiento del algoritmo durante las primeras iteraciones?

8. Entrega

Subir a **BrightSpace** un PDF con:

- ☐ Captura de pantalla de la configuración completa del task *Optimize*
 - ☐ Captura de las gráficas generadas (*Best value*, *Current value*, *Temperature*)
 - ☐ Captura de los resultados (x^* , y^* , f^*)
 - ☐ Resultados de al menos 2 experimentos del apartado 6
 - ☐ Respuestas a las preguntas de reflexión (mínimo 3 párrafos)
-

Referencias

- MathWorks. (2024). *simulannealbnd* — Minimize function with bounds using simulated annealing. MATLAB Documentation. <https://www.mathworks.com/help/gads/simulannealbnd.html>
 - MathWorks. (2024). *Optimize Live Task*. MATLAB Documentation. <https://www.mathworks.com/help/optim/ug/optimize-live-task.html>
 - Kirkpatrick, S., Gelatt, C. D., & Vecchi, M. P. (1983). Optimization by simulated annealing. *Science*, 220(4598), 671–680.
-

Documento preparado para uso académico — Fundamentos de Inteligencia Artificial

Dr. Aboud Barsekh - Onji, Facultad de Ingeniería, Universidad Anáhuac México